Tables as Visualization

Workshop 3 — Module 9

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

Module Roadmap

1. Table vs chart: when to use each
2. Five table design principles
3. Conditional formatting: colour-encoding in cells
4. Sparklines: Tufte's inline data graphics
5. R: `gt` and `kableExtra`
6. Python: `great_tables` and `pandas Styler`

Learning Outcome
You will produce publication-quality styled tables with conditional colour, inline sparklines, and proper typographic design in both R and Python.

Table vs Chart

When to Use a Table

Table vs Chart: When Each is Optimal

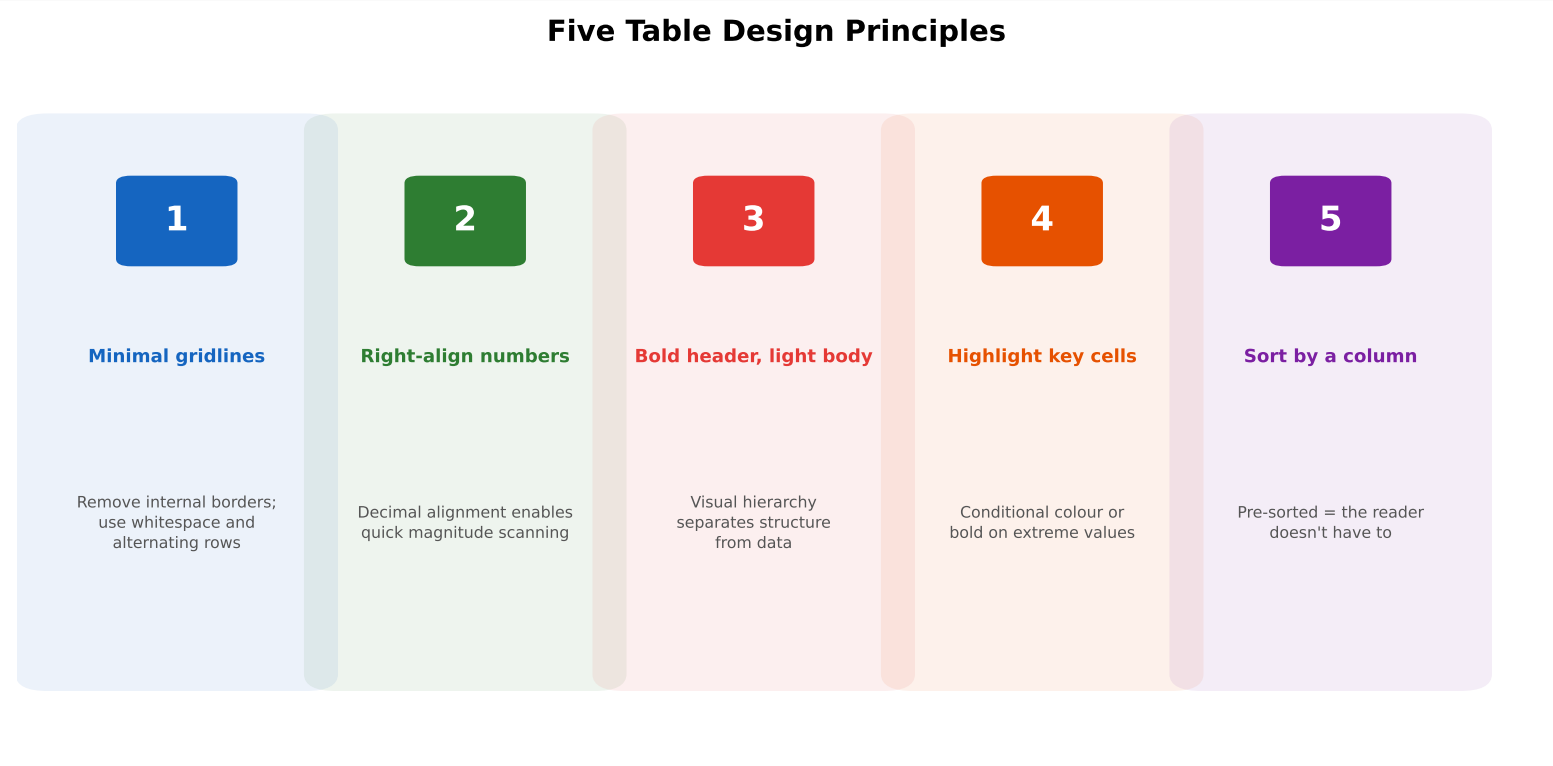
Situation	Use	Why
Exact values matter (financial, regulatory)	TABLE	$1,234,567.89 not ≈$1.2M
Pattern / trend matters	CHART	Trend visible instantly via position encoding
Few rows (≤20), many columns	TABLE	Grid layout is compact
Many rows (>20), few columns	CHART	Scroll fatigue; chart summarises
Lookup specific value	TABLE	Row/column intersection
Compare magnitudes	CHART	Length > reading numbers

Pro-Tip

Tables are best when the reader needs **exact values** or **lookup by row/column**. Charts are best when the reader needs to see **patterns, trends, or comparisons**. When in doubt, ask: "Does the reader need the number itself, or the pattern the numbers form?"

Table Design Principles

Five Design Principles



Plain vs Styled

Plain vs Styled: Design Transforms Readability

Plain Table
(default output, hard to scan)

Category	Count	Mean Rating	Median
FAMILY	1,832	4.18	4.30
GAME	959	4.05	4.10
TOOLS	843	4.12	4.20
BUSINESS	420	4.22	4.30
MEDICAL	395	3.95	4.00

Styled Table (gt / kableExtra)
(header, stripes, highlight)

Category	Count	Mean Rating	Median
FAMILY	**1,832**	4.18	4.30
GAME	959	4.05	4.10
TOOLS	843	4.12	4.20
BUSINESS	420	4.22	4.30
MEDICAL	395	3.95	4.00

Conditional Formatting

Colour Encoding Within Cells

Conditional Formatting: Colour Encodes Values Within Cells

Category	Mean Rating	Count	Paid %
FAMILY	**4.18**	1,832	73%
GAME	**4.05**	959	68%
TOOLS	**4.12**	843	71%
BUSINESS	**4.22**	420	82%
MEDICAL	**3.95**	395	65%

Data Insight

Conditional formatting bridges the table–chart gap: cells are coloured by value, enabling pattern detection while preserving exact numbers. Use sequential colour (light→dark) for magnitude; diverging colour (red–white–green) for deviation from a reference.

Conditional Formatting in R vs Python

R (gt)

```r
library(gt)

df |>
  gt() |>
  tab_header(
    title = "Category Summary",
    subtitle = "Google Play Store
        ") |>

  # Colour-encode Rating column
  data_color(
    columns = mean_rating,
    palette = "RdYlGn") |>

  # Format numbers
  fmt_number(columns = count,
    decimals = 0,
    use_seps = TRUE) |>
  fmt_percent(columns = paid_pct,
    decimals = 0) |>

  # Source note
  tab_source_note(
    "Source: Kaggle") |>
```

Python (pandas Styler)

```python
(df.style
    # Colour-encode Rating
    .background_gradient(
        subset=["mean_rating"],
        cmap="RdYlGn",
        vmin=3.8, vmax=4.4)

    # Bar chart in Count column
    .bar(subset=["count"],
        color="#BBDEFB")

    # Format
    .format({
        "count": "{:,.0f}",
        "mean_rating": "{:.2f}",
        "paid_pct": "{:.0%}"})

    # Highlight max
    .highlight_max(
        subset=["mean_rating"],
        color="#C8E6C9")

    # Caption
    .set_caption(
```

Sparklines

Sparklines: Inline Data Graphics

Sparklines in Tables: Tufte's Inline Data Graphics

Category	Trend (2019–2024)	Current	Change
FAMILY		4.18	+0.38
GAME		4.05	-0.15
TOOLS		4.12	+0.22
BUSINESS		4.22	+0.22
MEDICAL		3.95	-0.15

Pro-Tip

Tufte (2006) defined sparklines as "data-intense, design-simple, word-sized graphics." They embed a trend directly in the table row, eliminating the need for a separate chart. In R: `gt::gt_plt_sparkline()` or `kableExtra` with inline SVG. In Python: `great_tables` nanoplot or matplotlib inline rendering.

Sparklines in R (gt)

```r
library(gt)

df |>
  # Nest trend data as list column
  group_by(category) |>
  summarise(
    current = last(rating),
    trend = list(rating),          # list column of vectors
    change = last(rating) - first(rating),
    .groups = "drop") |>
  gt() |>
  # Render sparklines from list column
  gt_plt_sparkline(
    column = trend,
    type = "ref_line",             # add reference line at mean
    same_limit = TRUE,             # shared y range across rows
    palette = c("#1565C0",         # line colour
                "#1565C0",         # area fill
                "#E53935",         # reference line
                "#E53935",         # point high
                "#2E7D32")) |>     # point low
  fmt_number(columns = current, decimals = 2) |>
  tab_header(title = "Rating Trends by Category")
```

Styled Tables in Python (great_tables)

```python
from great_tables import GT, loc, style

(GT(df)
    .tab_header(
        title="Category Summary",
        subtitle="Google Play Store")

    # Colour-encode
    .data_color(
        columns="mean_rating",
        palette="RdYlGn")

    # Nanoplot sparklines (GT 0.6+)
    .fmt_nanoplot(
        columns="trend",
        plot_type="line")

    # Format
    .fmt_number(columns="count",
        decimals=0, use_seps=True)
    .fmt_percent(columns="paid_pct",
        decimals=0)

    # Source note
    .tab_source_note("Source: Kaggle")
)
```

Ecosystem

Table Packages: R & Python

Table Visualization Packages: R & Python

Package	Lang	Approach	Strength
`gt`	R	Declarative grammar, pipe-friendly API	Most polished output (HTML, LaTeX, RTF)
`kableExtra`	R	knitr::kable() extensions for RMarkdown	Quick and simple for reports
`great_tables`	Python	gt-inspired Python port	New (2024), growing ecosystem
`pandas Styler`	Python	df.style.background_gradient() .bar().format()	Built into pandas, Jupyter-native

kableExtra: Quick Tables for RMarkdown

```r
library(kableExtra)

df |>
  kable(format = "html",
    col.names = c("Category","Count","Rating","Paid %"),
    align = c("l","r","r","r"),          # right-align numbers
    digits = c(0, 0, 2, 0)) |>
  kable_styling(
    bootstrap_options = c("striped","hover","condensed"),
    full_width = FALSE,
    font_size = 12) |>
  row_spec(0, bold = TRUE,               # bold header
    color = "white", background = "#1565C0") |>
  column_spec(2, bold = TRUE) |>         # bold Count column
  row_spec(1, bold = TRUE,               # highlight top row
    background = "#E3F2FD") |>
  scroll_box(height = "400px")           # scrollable
```

Complete Examples

R: gt Styled Summary

```r
apps |>
  group_by(Category) |>
  summarise(
    Count = n(),
    `Mean Rating` = mean(Rating),
    `Paid %` = mean(Type == "Paid")
        ,
    .groups = "drop") |>
  slice_max(Count, n = 5) |>
  gt() |>
  tab_header(
    title = "Top 5 Categories",
    subtitle = "Google Play Store")
        |>
  data_color(
    columns = `Mean Rating`,
    palette = "RdYlGn") |>
  fmt_number(columns = Count,
    use_seps = TRUE) |>
  fmt_percent(columns = `Paid %`,
    decimals = 0) |>
  tab_source_note("Source: Kaggle")
      |>
  opt_row_striping()
```

gt: Styled Summary Table with Change Indicators

Category	Count	Rating	Change
FAMILY	1,832	4.18	▲ +0.38
GAME	959	4.05	▼ -0.15
TOOLS	843	4.12	▲ +0.22
BUSINESS	420	4.22	▲ +0.22
MEDICAL	395	3.95	▼ -0.15

Python: pandas Styler

```
summary = (apps
    .groupby("Category")
    .agg(Count=("Rating","count"),
         Rating=("Rating","mean"),
         Share=("Rating",
           lambda x: len(x)/len(
               apps)))
    .nlargest(5, "Count"))

(summary.style
    .background_gradient(
        subset=["Rating"],
        cmap="RdYlGn",
        vmin=3.8, vmax=4.4)
    .bar(subset=["Count"],
        color="#BBDEFB", vmin=0)
    .format({
        "Count": "{:,.0f}",
        "Rating": "{:.2f}",
        "Share": "{:.0%}"})
    .set_caption(
        "Top 5 Categories")
    .set_table_styles([
        {"selector": "caption",
         "props": [
```

pandas Styler: .background_gradient() on Rating

Category	Count	Rating	Share
FAMILY	1,832	4.18	38%
GAME	959	4.05	20%
TOOLS	843	4.12	18%
BUSINESS	420	4.22	9%
MEDICAL	395	3.95	8%

Synthesis

Key Takeaways

1. Use **tables for exact values**, **charts for patterns**. When the number itself is the message, a table wins.
2. Five design principles: **minimal gridlines**, **right-align numbers**, **bold header**, **highlight key cells**, **sort by a column**.
3. **Conditional formatting** bridges table and chart: colour-encoded cells enable pattern detection while preserving exact values.
4. **Sparklines** embed word-sized trend graphics directly in table rows (Tufte, 2006).
5. In R: `gt` (declarative, polished) or `kableExtra` (quick, RMarkdown-friendly). In Python: `great_tables` (gt-inspired) or `pandas Styler` (built-in).

Essential Readings

- Tufte, E. R. (2006). *Beautiful Evidence*, Chapter 2 (Sparklines).
- Few, S. (2012). *Show Me the Numbers*, Chapters 6–7 (Table Design).
- Iannone, R. et al. (2024). gt: Easily Create Presentation-Ready Tables. `https://gt.rstudio.com`
- Iannone, R. (2024). great_tables for Python. `https://posit-dev.github.io/great-tables/`

Next Module: Lab: Chart Selection Decision Framework (W03-M10)